POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

A lightweight canonicalization engine in C

WILLIAM ZHANG
Département de Electrical Engineering

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Electrical Engineering

July 2026

© William Zhang, 2026.

POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

Ce mémoire intitulé :

A lightweight canonicalization engine in C

présenté par **William ZHANG**
en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

Danielle DUBOIS, présidente
Sébastien VIGOT, membre et directeur de recherche
Marie COUTURE, membre et codirectrice de recherche
Jean TREMBLAY, membre
Joseph BROWN, membre externe

DEDICATION

À tous mes amis du labos,
vous me manquerez...

ACKNOWLEDGEMENTS

Texte / Text.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE OF CONTENTS

DEDICATION iii

ACKNOWLEDGEMENTS iv

RÉSUMÉ v

ABSTRACT vi

LIST OF TABLES ix

LIST OF FIGURES x

LIST OF SYMBOLS AND ACRONYMS xi

LIST OF APPENDICES xii

CHAPTER 1 INTRODUCTION 1
1.1 Historical background 1
1.1.1 Algebraic modeling languages 1
1.2 Conic optimization 3
1.3 Embedded modeling languages 4
1.3.1 `JuMP` and `MathOptInterface` 4
1.3.2 `YALMIP` 5
1.3.3 `CasADi` 5
1.4 Disciplined convex programming 6
1.4.1 `CVXPY` 6
1.4.2 `CVX` 8
1.4.3 `CVXR` 8
1.5 Problem statement 8
1.6 Research objectives 8
1.7 Thesis outline 8

CHAPTER 2 LITERATURE REVIEW 9
2.1 A Survey of Extraction Methods 9
2.1.1 `JuMP` and `MathOptInterface` 9
2.1.2 `CVXPY` 11
2.1.3 `CasADi` 14

2.1.4 `AMPL` . . . 14
2.2 A Jacobian Interpretation . . . 15
2.2.1 Properties of the affine Jacobian . . . 15
2.2.2 Connections to automatic differentiation . . . 15
2.2.3 The sparse composition bottleneck . . . 16
2.3 Summary and Research Gap . . . 16
2.4 Parameters in Optimization Problems . . . 17

CHAPTER 3 A LIGHTWEIGHT COEFFICIENT EXTRACTION ENGINE . . . 18
3.1 Design principles . . . 18
3.1.1 Architecture overview . . . 19
3.2 Worked example: tracing coefficient extraction . . . 19
3.3 Affine atoms . . . 21
3.3.1 Direct encoding of linear operators . . . 22
3.3.2 The `sum` atom and the `idx_map` trick . . . 22
3.3.3 The `left_matmul` atom and Kronecker avoidance . . . 24
3.3.4 The `index` atom and structural row selection . . . 25
3.4 Memory management . . . 26
3.5 Parameter handling . . . 27

CHAPTER 4 BENCHMARK RESULTS . . . 29

CHAPTER 5 Benchmark results . . . 30

CHAPTER 6 CONCLUSION . . . 31
6.1 Synthèse des travaux / Summary of Works . . . 31
6.2 Limitations de la solution proposée / Limitations . . . 31
6.3 Améliorations futures / Future Research . . . 31

REFERENCES . . . 32

APPENDICES . . . 36

LIST OF TABLES

Table 3.1 Catalogue of affine atoms. Each row describes one node type, its mathematical operation, and the core CSR transformation applied to the child's Jacobian during `eval_jacobian`. 21

LIST OF FIGURES

Figure 2.1 Expression tree for the affine expression `sum(A @ x - b) + C @ y`. Each internal node is an affine atom; leaves are variables or constants. 11

Figure 2.2 Coefficient matrix accumulation for `sum(A @ x)`. Each node composes its operator with the child's coefficient matrix via sparse matrix multiplication, producing a new intermediate matrix at every step. 12

Figure 3.1 CSR representation before and after applying `sum`. The 3-row matrix collapses to 1 row by setting `indptr` $= [0, 6]$. Duplicate column indices (0 and 2) represent values that sum together. The `indices` and `data` arrays remain unchanged—no matrix multiplication required. 22

LIST OF SYMBOLS AND ACRONYMS

AD	Automatic Differentiation
AML	Algebraic Modeling Language
CSC	Compressed Sparse Column
CSR	Compressed Sparse Row
DAG	Directed Acyclic Graph
DCP	Disciplined Convex Programming
LP	Linear Program
MOI	MathOptInterface
MPS	Mathematical Programming System
NLP	Nonlinear Program
QP	Quadratic Program
SOCP	Second-Order Cone Program
SDP	Semidefinite Program

LIST OF APPENDICES

Appendix A Démo 36
Appendix B Encore une annexe / Another Appendix 37
Appendix C Une dernière annexe / The Last Appendix 38

CHAPTER 1 INTRODUCTION

1.1 Historical background

Solving an optimization problem on a computer requires translating the mathematical formulation into a data format that solvers can process. For linear programs, this means constructing the constraint matrix A, the right-hand side vector b, and the objective coefficients c that define the *standard form*,

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned} \tag{1.1}$$

Manual encoding In the early days of linear programming, analysts manually computed and encoded each coefficient onto punch cards. This process was tedious, error-prone and became impractical as linear programming found more and more applications in large-scale industrial problems [1].

MPS format IBM's Mathematical Programming System (MPS) file format, developed in the 1960s, established a standardized file format for representing linear programs [2]. The MPS format uses fixed-width fields designed for 80-column punch cards, with sections for rows, columns, and coefficients. William Orchard-Hays [3] provides a comprehensive history of these early mathematical programming systems, tracing their development from 1952 at RAND Corporation through their widespread adoption on mainframe computers in the late 1960s.

Matrix generators To automate the creation of MPS files, *matrix generators* emerged in the late 1960s; they are procedural programs that compute the problem coefficients through explicit loops and output the results in MPS format [4]. Commercial systems such as `MaGen` [5], `GAMMA` [6], and `DATAFORM` [7] were among the first matrix generators which were used in various domains.

1.1.1 Algebraic modeling languages

Modelers naturally think in terms of sets, indices, and algebraic expressions, not in terms of individual matrix entries. Matrix generators simplified the encoding process but still required

modelers to translate their mathematical formulations into procedural code that explicitly constructs the coefficient matrices. This suggests that the gap between the modeler's representation of the problem and the solver's required format remained unresolved [1].

In the late 1970s, *algebraic modeling languages* emerged as formal languages designed to accept problem descriptions that closely mirror standard mathematical notation [8]. By working directly with the modeler's representation, these languages simplify verification (the model can be inspected as written), modification (changing the problem does not require rewriting procedural code), and communication (the formulation serves as its own documentation).

`GAMS`

`GAMS` (General Algebraic Modeling System) was developed in the early 1980s at the World Bank by Alexander Meeraus, Anthony Brooke, and David Kendrick [9]. Designed originally for large-scale economic planning models, `GAMS` introduced a set-based algebraic notation that maps naturally to the indexed summations and constraints common in economics and operations research. It was one of the first systems to cleanly separate model logic from data, allowing the same formulation to be solved with different datasets. A third major standalone Algebraic Modeling Language (AML), `AIMMS` [10], later extended this paradigm with a graphical development environment and end-user interfaces.

`AMPL`

`AMPL` (A Mathematical Programming Language) was created by Robert Fourer, David Gay, and Brian Kernighan at Bell Labs in the late 1980s [11]. Its declarative syntax was designed to mirror standard mathematical notation as closely as possible, making formulations directly readable by domain experts. A key design innovation is the strict separation of *model* (the algebraic structure) from *data* (the numerical values), allowing the same model file to be instantiated with different datasets.

Internally, `AMPL`'s compiler parses the model into an expression Directed Acyclic Graph (DAG) and writes the result to a `.nl` file [12], a compact binary or text format that encodes the expression graph along with coefficient data for linear and quadratic subexpressions. Solvers read this file directly, so the `.nl` format serves as the interface between the modeling and solving layers. More recently, the `AMPL/mp` solver library has been open-sourced, broadening community access to `AMPL`'s reformulation machinery.

1.2 Conic optimization

Interior-point methods The theoretical foundations for conic optimization were laid in the 1980s and 1990s. Karmarkar's 1984 polynomial-time interior-point algorithm for linear programming [13] demonstrated that Linear Programs (LPs) could be solved in polynomial time by traversing the interior of the feasible region rather than its vertices. Nesterov and Nemirovski [14] generalised interior-point methods to *self-concordant barrier* functions over convex cones, establishing polynomial complexity for a broad class of conic programs including Second-Order Cone Programs (SOCPs) and Semidefinite Programs (SDPs). These results made conic solvers practical: once the problem data is in the right form, the solver provides worst-case polynomial-time guarantees.

Cone types A *conic program* optimises a linear objective over the intersection of an affine subspace with a convex cone $\mathcal{K}$ [15]:

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & b - Ax \in \mathcal{K}, \end{array} \tag{1.2}$$

where $\mathcal{K}$ is a closed convex cone. The choice of $\mathcal{K}$ determines the problem class:

- **Nonnegative orthant** $\mathbf{R}^n_+ = \{x \in \mathbf{R}^n : x_i \geq 0\}$ — recovers linear programming.
- **Second-order cone** $\mathcal{Q}^n = \{(t, x) \in \mathbf{R} \times \mathbf{R}^{n-1} : \|x\|_2 \leq t\}$ — SOCPs, which capture quadratic constraints and ℓ_2-norm objectives.
- **Positive semidefinite cone** $\mathbf{S}^n_+ = \{X \in \mathbf{S}^n : X \succeq 0\}$ — SDPs, widely used in control, combinatorial optimisation, and signal processing.
- **Exponential and power cones** — extend the framework to entropy, geometric, and p-norm problems.

Conic form as a universal solver interface Modern conic solvers such as `MOSEK` [16,17] and `SCS` [18,19] accept problem data directly in the form $(A, b, c, \mathcal{K})$. This creates a clean separation of concerns: the *modeling layer* is responsible for reformulating the user's problem and extracting the coefficient matrices, while the *solver* handles the numerical optimisation. This thesis focuses on the modeling-layer side of the interface—specifically, how the sparse matrices A, b, and c are extracted from a canonicalized expression tree.

1.3 Embedded modeling languages

In the late 2000s and early 2010s, a new generation of modeling languages emerged that are *embedded* in general-purpose programming languages, in contrast with standalone systems like `AMPL` and `GAMS`. By living inside a host language, these tools inherit its full ecosystem: data manipulation libraries, scripting capabilities, testing frameworks, and package managers. Examples include `YALMIP` [20] in MATLAB, `Pyomo` [21] in Python, `CasADi` [22] in C++, and `JuMP` [23] in Julia. This section presents `JuMP` in most detail, as its `MathOptInterface` abstraction illustrates a particularly influential design for communicating structured problem data to solvers, then briefly covers `YALMIP` and `CasADi`.

1.3.1 `JuMP` and `MathOptInterface`

`JuMP` is an algebraic modeling language embedded in Julia, originally developed in 2012 by Iain Dunning, Joey Huchette, and Miles Lubin at the MIT Operations Research Center [23, 24]. Its abstraction layer, `MathOptInterface` (`MOI`) [25], rethinks how a modeling language communicates with solvers, moving away from the traditional approach of reducing every problem to a single standard form.

Function-set pairs `MOI` represents each constraint as a *function-set pair*: a mathematical function paired with the set it must belong to. For example, a linear inequality $a^T x \leq b$ is encoded as:

```
MOI.add_constraint(model,
    MOI.ScalarAffineFunction(MOI.ScalarAffineTerm.(a, x), 0.0),
    MOI.LessThan(b))
```

while a second-order cone constraint $t \geq \|x\|_2$ with n variables uses:

```
MOI.add_constraint(model,
    MOI.VectorOfVariables([t; x]),
    MOI.SecondOrderCone(n)).
```

`MOI` therefore preserves the semantic type of each constraint, so solvers that accept specific constraint types can receive them directly without unnecessary reformulations.

Bridges When a solver does not natively support a particular function-set pair, `MOI` applies a *bridge*, a composable transformation to an equivalent representation the solver accepts. The bridge system maintains a directed hyper-graph of known transformations and finds the

shortest path to the target solver's supported types, so adding a reformulation once benefits all solvers.

Incremental problem construction Some solvers, notably `MOSEK` [16] and `HiGHS` [26], allow constraints to be added one at a time rather than requiring the entire problem to be submitted at once. `MOI` supports this incremental mode natively, so algorithms that iteratively refine a formulation, such as cutting-plane or branch-and-cut methods, can append a constraint between successive solves without reconstructing the full problem data.

Extensibility `MOI` makes it straightforward to support new constraint types: a solver declares which function-set pairs it handles, and new bridges can be registered at any time. `Hypatia` [27, 28], for example, natively implements over twenty specialized cones, including the log-determinant cone, the relative-entropy cone, and several spectral matrix cones, all of which are first-class `MOI` set types. `SCS` [18,19] has similarly begun adding native spectral matrix cone support [29], bypassing the large semidefinite reformulations that would otherwise be required.

1.3.2 `YALMIP`

`YALMIP` [20] is an optimization modeling toolbox for MATLAB, developed by Johan Löfberg starting in 2004. It overloads MATLAB's arithmetic operators on *sdpvar* decision-variable objects, building an internal expression graph as the user writes natural mathematical expressions. When the user calls `optimize()`, `YALMIP` analyses the expression graph to automatically detect the problem class—LP, Quadratic Program (QP), SOCP, SDP, or mixed-integer variant—and dispatches to an appropriate solver. Unlike DCP-based tools, `YALMIP` does not enforce convexity rules at construction time; instead, it applies a broader set of reformulation strategies to handle problems that may not fit the DCP framework, at the cost of weaker compile-time guarantees.

1.3.3 `CasADi`

`CasADi` [22] is an open-source framework for nonlinear optimisation and optimal control, developed by Joel Andersson, Joris Gillis, and Moritz Diehl. Written in C++ with Python and MATLAB interfaces, it builds a symbolic DAG of the problem expressions and uses Automatic Differentiation (AD) to compute exact derivatives. Its primary targets are Nonlinear Program (NLP) solvers such as `IPOPT` [30], making it the tool of choice for model-predictive control and trajectory optimisation. For the special case of affine expressions, the Jacobian

$\partial f/\partial x = A$ is exactly the coefficient matrix, so `CasADi`'s AD machinery performs coefficient extraction as a by-product of its general differentiation pipeline.

1.4 Disciplined convex programming

A complementary approach to embedded modeling is *disciplined convex programming* (DCP) [31, 32], a set of composition rules that can automatically verify convexity. A DCP language restricts how atoms (elementary functions with known curvature) may be combined: for example, a convex function of an affine expression is convex, but a convex function of a convex expression is not necessarily so. By enforcing these rules at problem construction time, DCP languages guarantee that any accepted formulation is convex and can be reduced to conic standard form. Implementations include `CVX` [33] in MATLAB, `CVXPY` [34, 35] in Python, `CVXR` [36] in R, and `Convex.jl` [37] in Julia. This section presents `CVXPY` in most detail, as it is the system targeted by the coefficient extraction engine developed in this thesis, then briefly covers `CVX` and `CVXR`. `Convex.jl` follows a similar DCP pipeline in Julia and is not discussed further.

1.4.1 `CVXPY`

`CVXPY` is a domain-specific language for convex optimization embedded in Python. It was originally developed by Steven Diamond and Stephen Boyd at Stanford University [34]. Akshay Agrawal later redesigned the rewriting system in `CVXPY` [35], which enabled support for other disciplined languages [38–40]. The project has since grown through contributions from many other developers and solver interface maintainers.

Reduction-based rewriting `CVXPY` is built on a system of *reductions* [35]. A reduction is a pair of maps: a *forward* map that transforms a problem into an equivalent one, and a *backward* map that recovers a solution of the original problem from a solution of the transformed one. Reductions compose: the output of one becomes the input of the next, forming a *chain* that converts the user's formulation into solver-ready data.

The chain is solver-aware, so different solvers trigger different output formats. Historically, `CVXPY` distinguished two standard forms. For solvers that accept quadratic objectives such as `OSQP` and `Gurobi`, the chain produces a quadratic program:

$$\begin{aligned} &\text{minimize} && \tfrac{1}{2}\, x^T P\, x + c^T x \\ &\text{subject to} && l \le Ax \le u. \end{aligned} \tag{1.3}$$

For purely conic solvers (*e.g.*`SCS`), the chain instead produces:

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & Ax + b \in \mathcal{K}. \end{array} \tag{1.4}$$

More recently, a unified interface that combines a quadratic objective with conic constraints has become the central solver interface in `CVXPY`:

$$\begin{array}{ll} \text{minimize} & \frac{1}{2}\, x^T P\, x + c^T x \\ \text{subject to} & Ax + b \in \mathcal{K}. \end{array} \tag{1.5}$$

In all cases the final output is a set of sparse coefficient matrices; what changes between reduction paths is the *structure* of those matrices.

Common reductions The individual reductions that compose a canonicalization chain are well established in the convex optimization literature. Chu et al. [41] give a concise exposition; the main steps are:

1. **DCP verification.** Confirm that the objective is convex and the constraints are valid by walking the expression tree and applying DCP composition rules.
2. **Smith form.** Replace each subexpression with a new auxiliary variable linked to the original expression by an equality constraint.
3. **Relaxed Smith form.** Relax the equalities introduced above to inequalities wherever the monotonicity of the objective guarantees that the inequality will be tight at optimality.
4. **Matrix stuffing.** Assemble the resulting affine expressions into sparse coefficient matrices before passing them to a solver.

The following chapter examines how these steps—and in particular the final matrix-stuffing step—are implemented in practice across different systems.

Other reductions `CVXPY` also ships reductions beyond the standard canonicalization chain. For example, a *flip-objective* reduction converts a maximization problem into a minimization, and the `Dgp2Dcp` reduction rewrites disciplined geometric programs into equivalent disciplined convex programs [38].

1.4.2 CVX

CVX [33] is the original implementation of disciplined convex programming, developed by Michael Grant and Stephen Boyd at Stanford University starting in 2004 [31]. Embedded in MATLAB, it uses cvx_begin/cvx_end blocks to delimit a problem specification, within which MATLAB expressions are intercepted and checked against DCP rules. Grant's PhD thesis [31] formalised the DCP ruleset and proved that any DCP-compliant problem can be reduced to conic standard form. On the solver side, CVX targets SeDuMi and SDPT3 by default, producing the SDP data structures these solvers expect. CVX demonstrated that a modeling language could both verify convexity and automate the reduction to solver-ready form, establishing the template that CVXPY and subsequent DCP tools follow.

1.4.3 CVXR

CVXR [36] brings disciplined convex programming to the R statistical computing environment. Developed by Anqi Fu, Balasubramanian Narasimhan, and Stephen Boyd, it uses R's S4 object system to overload arithmetic operators on decision variables, building an expression tree that is checked against DCP rules. CVXR targets the same solver backends as CVXPY (including SCS and MOSEK) and follows the same canonicalization-then-extraction pipeline. Its existence in R demonstrates the portability of the DCP paradigm: the same composition rules and reduction strategy can be implemented across languages with different type systems and user communities.

1.5 Problem statement

1.6 Research objectives

1.7 Thesis outline

CHAPTER 2 LITERATURE REVIEW

2.1 A Survey of Extraction Methods

Solvers require problem data in a standard form: the constraint matrix A, right-hand side b, objective coefficients c, and a cone description $\mathcal{K}$. Modeling languages let users write natural mathematical expressions. Something must bridge the gap.

This "something" involves two logically distinct operations:

1. **Reformulation.** Transform the problem into an equivalent standard-form problem by introducing auxiliary variables, epigraph constraints, and conic representations. Chapter 1 described how `CVXPY`'s reduction chain and `MOI`'s bridge system each accomplish this.

2. **Coefficient extraction.** Extract the numerical coefficient data (A, b, c) from the reformulated—now affine—expressions.

The literature has largely treated these as a single pipeline. This section examines them separately, focusing on the second operation: how different systems actually compute the sparse matrices that solvers consume.

To make the comparison concrete, we trace a single running example through each system: the least-squares problem

$$\text{minimize} \quad \|Fx - g\|_2^2, \tag{2.1}$$

with $F \in \mathbf{R}^{3\times 2}$ and $g \in \mathbf{R}^3$.

2.1.1 `JuMP` and `MathOptInterface`

`JuMP`'s function-in-set architecture, described in Chapter 1, uses operator overloading to eagerly build coefficient representations. There is no expression tree and no deferred extraction phase—the coefficients *are* the internal representation. The following trace shows how this works for the least-squares problem (2.1).

Step 1: Macro expansion The user writes:

```
using JuMP, MosekTools
F = [1 2; 3 4; 5 6]; g = [1, 2, 3]
```

```
model = Model(Mosek.Optimizer)
@variable(model, x[1:2])
@objective(model, Min, sum((F * x - g).^2))
```

The `@objective` macro expands `sum((F * x - g) .^ 2)` by evaluating each subexpression eagerly. The product `F * x` dispatches through Julia's linear algebra: each element $r_i = F_{i1}x_1 + F_{i2}x_2 - g_i$ is built as a `ScalarAffineFunction` containing the coefficients F_{i1}, F_{i2} and the constant $-g_i$ explicitly.

Step 2: Quadratic function construction Squaring each residual r_i^2 triggers `JuMP`'s multiplication dispatch, which collects the quadratic terms into a `ScalarQuadraticFunction`:

$$r_i^2 = F_{i1}^2 x_1^2 + 2F_{i1}F_{i2}\, x_1 x_2 + F_{i2}^2 x_2^2 - 2g_i(F_{i1}x_1 + F_{i2}x_2) + g_i^2.$$

Summing over $i = 1, 2, 3$ accumulates all quadratic and linear terms into a single `ScalarQuadraticFunction` representing $\sum_i r_i^2$.

Step 3: Implicit formation of F^TF The algebra reveals that by expanding the sum of squares, `JuMP` has implicitly formed the Gram matrix:

$$\|Fx - g\|_2^2 = x^T F^T F\, x - 2g^T F\, x + g^T g.$$

The quadratic coefficient matrix stored in the `ScalarQuadraticFunction` is $Q = F^TF$, the linear coefficient vector is $c = -2F^Tg$, and the constant is g^Tg. These are the actual numerical values, not symbolic expressions.

Step 4: Solver handoff `MOSEK` natively supports quadratic objectives, so the `MOI` bridge system finds no transformation is needed. The `ScalarQuadraticFunction`—containing Q, c, and the constant—is forwarded directly to the solver's API.

Consequence Compositions are resolved at expression-build time. There is no separate extraction phase because extraction happens incrementally during construction. But this means structure is lost: F^TF cannot be "un-formed." A conic solver that could accept $\|Fx - g\|_2$ directly as a second-order cone constraint would need to recover F from F^TF, which is not possible in general. When such a reformulation is needed, `MOI` activates a bridge, but the bridge operates on the already-formed quadratic representation rather than the original affine expression.

2.1.2 `CVXPY`

`CVXPY` takes the opposite approach: it builds an expression tree of atoms and defers all numerical work until the solver is invoked. The pipeline has two phases:

1. DCP-based canonicalization rewrites the tree into conic standard form (the reductions framework described in Chapter 1) [35].
2. *Matrix stuffing* walks the canonicalized tree and extracts (A, b, c) by composing sparse linear maps.

This section focuses on the second phase: how `CVXPY` actually computes the coefficient matrices from the canonicalized expression tree.

Expression trees After canonicalization, `CVXPY` guarantees that all constraints and the objective are affine functions of the decision variable x. Each affine function is represented as an *expression tree* where every node is a linear operation (an *atom*), as illustrated in Figure 2.1.

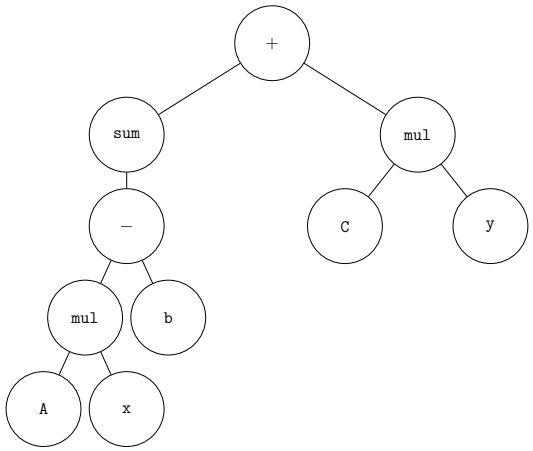


Figure 2.1 Expression tree for the affine expression `sum(A @ x - b) + C @ y`. Each internal node is an affine atom; leaves are variables or constants.

When a problem involves matrix variables, `CVXPY` flattens them into vectors using column-major (Fortran) ordering: $X \in \mathbf{R}^{m \times n}$ becomes $(X_{11}, X_{21}, \ldots, X_{m1}, X_{12}, \ldots, X_{mn})^T \in \mathbf{R}^{mn}$. This convention determines the column ordering in all coefficient matrices.

Matrix stuffing via sparse linear map composition The extraction phase traverses each expression tree bottom-up. Each leaf node returns its coefficient matrix (the identity

for a variable, a constant matrix for data), and each internal node composes its children's matrices by applying its own linear operator. The composition is carried out via sparse matrix multiplication.

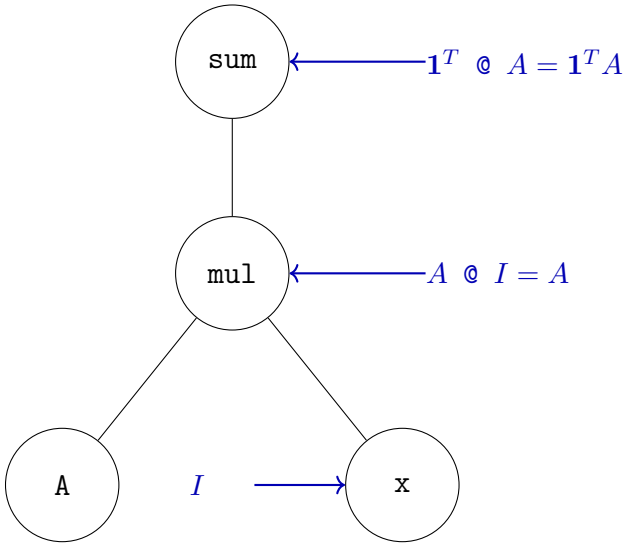


Figure 2.2 Coefficient matrix accumulation for `sum(A @ x)`. Each node composes its operator with the child's coefficient matrix via sparse matrix multiplication, producing a new intermediate matrix at every step.

For example, in the expression `sum(A @ x)`, the traversal first returns I at the variable node, multiplies by A at the `mul` node to get A, then multiplies by $\mathbf{1}^T$ at the `sum` node to get $\mathbf{1}^T A$ (Figure 2.2). Each step performs a sparse matrix–matrix product and allocates a new result matrix.

Architecture of the extraction backends Internally, `CVXPY` represents the canonicalized tree as a list of `LinOp` nodes. The extraction is handled by a `PythonCanonBackend` whose `build_matrix` method walks each tree and returns a 2D sparse matrix. The central data structure is a `TensorView`: a dictionary mapping each variable to its coefficient block (a sparse matrix). A recursive function `process_constraint` traverses the tree bottom-up, applying the appropriate linear operator at each node.

The operations performed during this traversal fall into three categories:

1. **Pure index operations** (`reshape`, `transpose`, `index`, `promote`, `broadcast_to`). These only remap row indices—no arithmetic is performed. Essentially free in terms of floating-point cost.

2. **Elementwise operations** (`neg`, `mul_elem`, `div`). These scale existing entries via elementwise multiplication. The sparsity pattern is unchanged.

3. **Matrix multiplication operations** (`mul`, `rmul`, `sum_entries`, `conv`, `kron`). These build an explicit operator matrix and apply it via general-purpose `scipy.sparse @ scipy.sparse`. *This is where the cost lives.*

Detailed example: left multiplication Consider the node `C @ X` where $C \in \mathbf{R}^{m \times n}$ is a constant and $X \in \mathbf{R}^{n \times p}$ is the child expression. The vectorised form is $\mathbf{vec}(CX) = (I_p \otimes C)\,\mathbf{vec}(X)$, so the backend builds the Kronecker product $I_p \otimes C$ via `sp.kron(sp.eye(p), C)`, applies it to the child's coefficient matrix via sparse matrix multiplication, and converts the result to CSR. This sequence involves three allocations per node: the Kronecker product, the matrix product, and the format conversion.

What is already exploited `CVXPY`'s implementation does exploit some structure. Index operations are handled as pure reindexing without forming operator matrices. Elementwise operations use `.multiply()` instead of matrix multiplication. The Kronecker structure is partially exploited by using `sp.kron(sp.eye(n), C)` rather than materialising a dense matrix.

What is not exploited Several opportunities for efficiency remain:

- The Kronecker operator $I_p \otimes C$ is materialised as a general sparse matrix, even though its block-diagonal structure means C could be applied independently to p blocks.
- Sparsity patterns are discovered at runtime by `SciPy`'s sparse arithmetic, even though they are fully determinable from the expression structure at build time.
- No pre-allocation of the final output is possible, since each intermediate product has an independently computed sparsity pattern.
- General-purpose sparse matrix multiplication is used regardless of the known structure of the operator (diagonal scaling, row selection, Kronecker with identity, *etc.*).

`CVXPY` ships two Python backends (SciPy-based and COO-based) as well as a legacy C++ backend (`cvxcore`) that special-cases eighteen `LinOp` types. All three share the same compositional architecture.

Key advantage of the deferred approach Unlike `JuMP`'s eager expansion, the deferred approach preserves the original problem structure through canonicalization. The expression `norm(F @ x - g)` never forms F^TF; the matrix F appears directly in the coefficient data. This structural preservation is what enables conic reformulations that exploit the original factors.

2.1.3 `CasADi`

`CasADi` [22] takes a third approach. It is fundamentally a nonlinear programming tool: its primary targets are NLP solvers such as `IPOPT` [30], with conic capabilities (QP, SOCP) available through external backends but not as a central design focus. Internally, `CasADi` builds a symbolic expression DAG and extracts derivative information via algorithmic differentiation on the graph. For an affine expression $f(x) = Ax + b$, the Jacobian $\partial f/\partial x = A$ is the coefficient matrix, so AD-based Jacobian extraction and coefficient extraction are mathematically identical. Crucially, `CasADi` propagates sparsity symbolically through the graph before performing numerical evaluation, making it a genuine hybrid of symbolic and AD-based approaches.

This is relevant because it explicitly connects expression-graph processing to Jacobian computation, bridging the modeling and differentiation worlds. However, since `CasADi` is designed for general nonlinear problems, it applies the same AD machinery to affine expressions as to nonlinear ones—it has no specialised fast path for the purely affine case where the Jacobian is constant and the coefficients could be read off structurally rather than computed through differentiation.

2.1.4 `AMPL`

`AMPL` [11] historically pioneered systematic coefficient extraction. Its internal representation is an expression DAG in which each node is classified by polynomial degree: constant, linear, quadratic, or general nonlinear [12]. Linear and quadratic coefficients are extracted directly from the classified nodes by walking the DAG, while the nonlinear remainder is retained as an evaluable subgraph for AD-based derivative computation. This structural decomposition allows `AMPL` to hand optimised coefficient data to linear and quadratic solvers while still supporting fully nonlinear formulations. However, `AMPL`'s extraction uses general-purpose operations for the linear and quadratic coefficients rather than structure-aware CSR transformations, and its closed-source commercial nature has limited community development of specialised alternatives.

2.2 A Jacobian Interpretation

The previous subsections described four different systems, each with its own architecture. Despite their differences, a common mathematical structure underlies them all: extracting the coefficient matrix A from an affine expression $f(x) = Ax + b$ is equivalent to computing the Jacobian $\partial f/\partial x = A$. The Jacobian is constant (independent of x), so one computation suffices for all x.

2.2.1 Properties of the affine Jacobian

Three properties distinguish this Jacobian from general Jacobian computation in numerical optimisation:

(a) **Constant.** Since f is affine, $\partial f/\partial x = A$ is independent of x. One evaluation produces the exact result for all inputs.

(b) **Structurally determined sparsity.** The sparsity pattern of A is fully determined at expression-build time, before any numerical values are known. Each atom's contribution to the sparsity pattern depends only on dimensions, index sets, and the topology of the expression tree.

(c) **Structured local Jacobians.** The expression is a composition of affine operations, each with a known, structured local Jacobian. These are not arbitrary sparse matrices: they are typically row scalings, permutations, block diagonals, row selections, or Kronecker products with the identity.

2.2.2 Connections to automatic differentiation

The coefficient extraction problem can, in principle, be solved by any method that computes Jacobians. Existing approaches from the differentiation literature each address part of the problem but leave efficiency on the table:

Symbolic differentiation For an affine function $f(x) = Ax + b$, symbolic differentiation correctly yields $\partial f/\partial x = A$. However, computer algebra systems produce symbolic expressions, not the sparse numerical arrays that solvers require. The output format does not match the needed representation.

Automatic differentiation Forward- and reverse-mode AD compute Jacobian-vector products (Jv) or vector-Jacobian products ($v^T J$), not the explicit Jacobian matrix. Extracting the full sparse matrix requires either n forward passes (one per column) or m reverse passes (one per row), or sparse compression via graph colouring [42]. This is general-purpose machinery applied to a special case where the answer is a constant matrix.

Optimal Jacobian accumulation Naumann [43, 44] studies the problem of efficiently ordering the multiplication of Jacobian factors in a composition chain. This is directly relevant: when $f = f_k \circ \cdots \circ f_1$, the chain rule gives $J = J_k \cdots J_1$, and the multiplication order affects the total cost. However, the optimal Jacobian accumulation literature treats each J_i as an arbitrary sparse matrix, without exploiting the structured forms (diagonal, permutation, block-diagonal) that arise in affine expression trees.

None of these methods exploit all three special properties simultaneously: the constancy of the Jacobian, the a priori knowledge of the sparsity pattern, and the structured form of each local Jacobian.

2.2.3 The sparse composition bottleneck

When $f = f_k \circ \cdots \circ f_1$ is a composition of affine maps, the overall Jacobian is the matrix product $A_k \cdots A_1$ (the chain rule specialised to constant Jacobians). `CVXPY` computes this product during matrix stuffing via repeated sparse matrix–matrix multiplication. Two costs arise:

1. **Intermediate matrix allocation.** Each partial product $A_k \cdots A_i$ is materialised as a new sparse matrix. For a tree of depth d, up to $d-1$ intermediate matrices are created and discarded.

2. **General-purpose sparse matrix multiplication.** Each multiplication treats A_i as an arbitrary sparse matrix, invoking `SciPy`'s general `csr_matrix @ csr_matrix` routine. This routine must discover the output sparsity pattern at runtime, allocate output arrays, and sort indices—overhead that is unnecessary when the structure of A_i is known in advance.

2.3 Summary and Research Gap

No existing implementation exploits all three properties of the affine Jacobian simultaneously:

(a) That local Jacobians are **structured operations** with known forms (row scaling, selection, block-diagonal multiplication, Kronecker with identity). Each composition step could be a direct transformation of the sparse storage arrays—inserting rows, scaling values, selecting entries—rather than a general-purpose sparse matrix multiplication.

(b) That the sparsity pattern of the output is **determinable before numerical evaluation**. The entire output data structure—row pointers, column indices, number of nonzeros—can be pre-allocated in a single pass over the expression tree, before any floating-point arithmetic occurs.

(c) That **no intermediate matrices need to be materialised**. The composition can be computed as a single pass that writes directly into the pre-allocated output arrays, transforming index and value arrays in-place rather than creating and discarding temporary matrices at each node.

This is the gap that the present work fills. The next chapter presents a coefficient extraction engine that exploits all three properties: it performs direct CSR array transformations for each atom, pre-computes sparsity patterns at build time, and writes results directly into pre-allocated output arrays without forming intermediate matrices.

2.4 Parameters in Optimization Problems

When a problem contains parameters, `CVXPY` compiles the structure once—sparsity pattern, cone types, variable ordering—and stores a mapping from parameter values to numerical coefficients. Subsequent solves only require evaluating this mapping and calling the solver, skipping the full canonicalization. The `cvxpygen` library [45] extends this by generating standalone C code embedding the compiled structure, enabling deployment without a Python runtime.

This compile-once, evaluate-many paradigm naturally motivates a two-phase engine design: an initialisation phase that determines the sparsity structure, and an evaluation phase that fills in numerical values. The next chapter adopts precisely this separation.

CHAPTER 3 A LIGHTWEIGHT COEFFICIENT EXTRACTION ENGINE

The previous chapter showed that coefficient extraction in modern modeling languages relies on generic sparse matrix multiplication—an approach that creates intermediate matrices at every node of the expression tree and treats each linear operator as an arbitrary sparse matrix. This chapter presents an alternative: a lightweight engine written in C that operates directly on CSR index arrays, avoiding intermediate matrices entirely.

3.1 Design principles

The engine is guided by three principles, each addressing a specific inefficiency identified in the previous chapter's analysis of existing systems.

Principle 1: Direct CSR rewriting Every affine atom transforms the CSR arrays `(indptr, indices, data)` of the child's Jacobian directly, rather than forming an operator matrix and composing via sparse matrix multiplication. A `sum` node merges row pointers; a `left_matmul` node applies block-diagonal dot products; an `index` node copies row segments. No intermediate sparse matrices are formed at any point.

Principle 2: Two-phase evaluation Sparsity patterns are computed once at build time (*init phase*); numerical values are filled at solve time (*eval phase*). The init phase traverses the expression tree bottom-up, determining the exact number of nonzeros in each Jacobian and allocating all arrays to their final sizes. The eval phase traverses the same tree, filling values into pre-allocated arrays without any allocation. This separation mirrors the compile-once, evaluate-many paradigm used in parametric optimisation (Section 2.4).

Principle 3: Precomputed index mappings Where rows must be combined, reordered, or selected, a mapping from source to destination positions is precomputed during init. At eval time, the operation reduces to a single $O(\texttt{nnz})$ scatter loop. The `sum` atom's `idx_map` array (Section 3.3.2) is the prototypical example: rather than performing a sparse matrix multiplication $\mathbf{1}^T J_x$ at every evaluation, the engine scatters the child's nonzeros through a fixed map.

3.1.1 Architecture overview

The engine represents each affine expression as a node in a directed acyclic graph (DAG). Each node is a C `struct` containing:

- the node's *type* (one of the fifteen atoms in Table 3.1),
- a pair of function pointers—`jacobian_init` and `eval_jacobian`—that implement the two-phase protocol for that atom type,
- pointers to child nodes (one or two, depending on the atom),
- the Jacobian in CSR format: arrays `indptr`, `indices`, and `data`, plus dimensions,
- atom-specific auxiliary data (index maps, constant matrices, axis parameters).

Function-pointer dispatch on the node type provides polymorphism without virtual-table overhead.

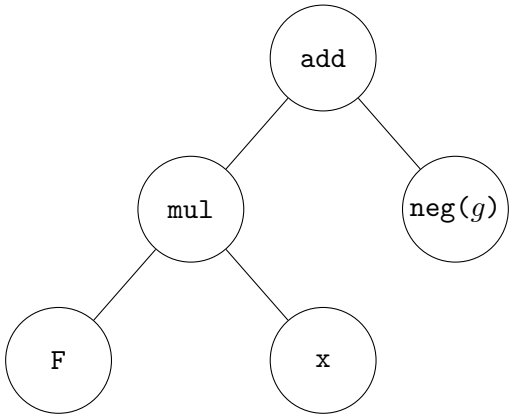
When two expressions share a common subexpression, the DAG representation allows the shared node's Jacobian to be computed once and reused by both parents. The engine tracks whether each node's Jacobian has been evaluated via a `jacobian_called` flag, avoiding redundant computation.

A *problem container* groups the objective expression and all constraint expressions, manages the global variable ordering, and stores the final assembled coefficient matrices (A, b, c) that are handed to the solver.

3.2 Worked example: tracing coefficient extraction

To build intuition before the atom-by-atom treatment, we trace the least-squares problem minimize $\|Fx - g\|_2^2$ from Chapter 2 end-to-end through the engine. After canonicalization (DCP verification, Smith form, and relaxed Smith form as described in Section 2), the conic reformulation produces the equality constraint $t = Fx - g$ linking the auxiliary variable t to the original expression. The engine must extract the coefficient matrix for this affine expression.

Expression tree after canonicalization The affine expression $Fx - g$ is represented as:



The `mul` node represents left multiplication by the constant matrix $F \in \mathbf{R}^{3\times 2}$, the `neg` node negates the constant $g \in \mathbf{R}^3$, and the `add` node sums the two children.

Init phase The engine traverses the tree bottom-up, computing sparsity patterns:

1. **Variable leaf** (x): Jacobian is I_2 (identity), so `nnz` $= 2$, `indptr` $= [0, 1, 2]$, `indices` $= [0, 1]$.
2. **`mul` node** ($F \cdot x$): The Jacobian with respect to x is F itself, a 3×2 matrix. Init determines `nnz` $= 6$ (all entries of F are nonzero) and allocates arrays of the corresponding sizes.
3. **Constant leaf** ($-g$): No variable dependence, contributes only to the constant column. Jacobian with respect to x is the 3×2 zero matrix.
4. **`add` node**: Takes the union of the children's sparsity patterns. Since the constant $-g$ contributes no variable columns, the output sparsity matches the `mul` child: `nnz` $= 6$.

After init, every node's CSR arrays are allocated to their exact final sizes. No further allocation occurs.

Eval phase A second bottom-up traversal fills in numerical values:

1. **Variable leaf**: `data` $= [1, 1]$ (identity values).
2. **`mul` node**: Applies F to the child's Jacobian via block-diagonal dot products (Section 3.3.3), writing the six entries of F into `data`.
3. **`add` node**: Sums corresponding entries. The constant offset $-g$ is added to the constant column of the output.

Result The root node's Jacobian is the coefficient matrix:

$$\begin{array}{ccc} x_1 & x_2 & 1 \\ \left[\begin{array}{cc|c} 1 & 2 & -1 \\ 3 & 4 & -2 \\ 5 & 6 & -3 \end{array}\right] & = [F \mid -g], \end{array}$$

which matches the constraint $t - Fx = -g$ from the canonicalization. The entire computation involved no sparse matrix multiplication, no intermediate matrix allocation, and no runtime sparsity discovery.

3.3 Affine atoms

Table 3.1 catalogues the fifteen affine atoms implemented in the engine. Each atom exposes two entry points: `jacobian_init`, which determines the output Jacobian's sparsity pattern once at problem-build time, and `eval_jacobian`, which fills in the numerical values each time parameters change. The table's third column summarises the key CSR manipulation performed by each atom.

Table 3.1 Catalogue of affine atoms. Each row describes one node type, its mathematical operation, and the core CSR transformation applied to the child's Jacobian during `eval_jacobian`.

Atom	Math operation	Jacobian action on CSR
`variable`	$y = x$	Identity (leaf node)
`parameter`	$y = \theta$	None (data node)
`linear_op`	$y = Ax + b$	$J = A$ (stored directly)
`add`	$y = a + b$	Union sparsity, sum values
`neg`	$y = -x$	Same sparsity, negate `data`
`sum`	$y = \sum x$ (along axis)	Merge rows via `idx_map`
`hstack`	$y = [a; b; \ldots]$	Concatenate row blocks
`index`	$y = x[\text{idx}]$	Select/reorder rows
`reshape`	$\text{reshape}(x, m, n)$	No-op (copy arrays)
`transpose`	$y = X^T$	Permute rows
`broadcast`	$(1, n) \to (m, n)$	Tile/replicate rows
`promote`	scalar $\to$ vector	Replicate single row
`trace`	$y = \mathbf{Tr}(X)$	Sum diagonal rows via `idx_map`
`diag_vec`	$v \to \mathbf{diag}(v)$	Place rows at diagonal positions
`left_matmul`	$y = A \cdot f(x)$	Block-diagonal multiply (no Kronecker)

The subsections that follow first describe the general principle of encoding linear operators directly on CSR arrays, then examine three representative atoms in detail: `sum` (Section 3.3.2),

which illustrates the `idx_map` accumulation trick; `left_matmul` (Section 3.3.3), which avoids forming a Kronecker product; and `index` (Section 3.3.4), which performs pure structural row selection with no arithmetic.

3.3.1 Direct encoding of linear operators

Rather than composing operators via sparse matrix multiplication, we keep the coefficient matrix in CSR (Compressed Sparse Row) format and directly manipulate its index arrays without forming intermediate matrices. Each linear operator is implemented as a transformation on the CSR arrays `(indptr, indices, data)`. For example, the `sum` operator collapses all rows into row 0 by setting all entries of `indptr` to $[0, \texttt{nnz}]$, effectively mapping every row index to 0 without any matrix multiplication.

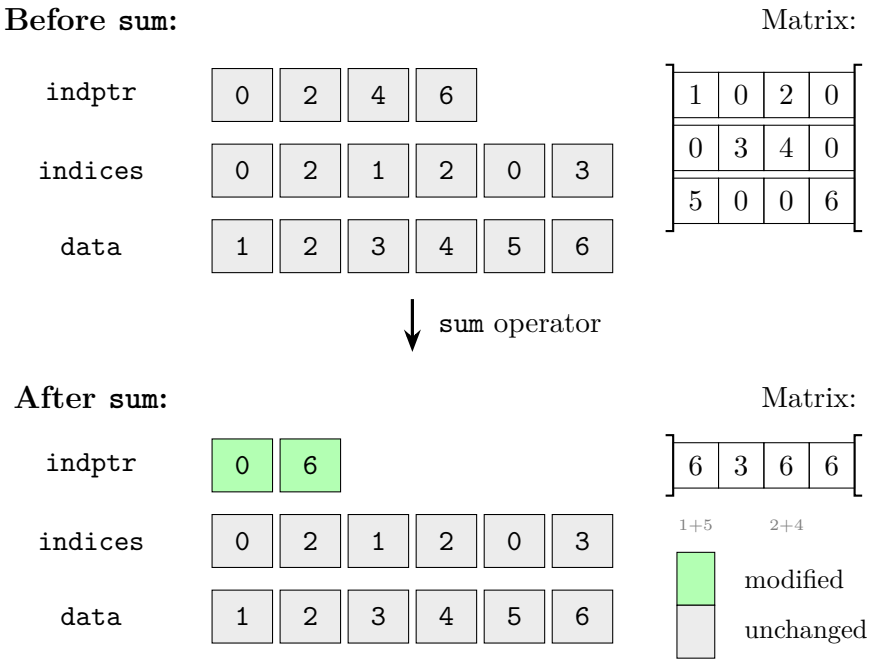


Figure 3.1 CSR representation before and after applying `sum`. The 3-row matrix collapses to 1 row by setting `indptr` $= [0, 6]$. Duplicate column indices (0 and 2) represent values that sum together. The `indices` and `data` arrays remain unchanged—no matrix multiplication required.

3.3.2 The `sum` atom and the `idx_map` trick

Summing along an axis collapses multiple rows of the child's Jacobian into fewer rows. Mathematically, for a full summation ($y = \mathbf{1}^T x$, scalar output), the Jacobian is $J_y = \mathbf{1}^T J_{\text{child}}$—every row of J_{child} is accumulated into a single output row. The naïve approach would compute

this via sparse matrix multiplication, allocating a new matrix each time. Instead, the engine precomputes a mapping from every nonzero position in the child's Jacobian to its target position in the output, then uses a simple scatter-accumulate loop at evaluation time.

Initialisation (Algorithm 1) During `jacobian_init`, the output sparsity pattern is determined by taking the union of all column indices that appear across the rows being summed. Duplicate column indices from different rows are merged into a single output entry. An array `idx_map` of length $\texttt{nnz}(J_{\text{child}})$ is then built: for each nonzero k in the child's Jacobian, `idx_map`$[k]$ stores the position in the output's `data` array where that value should be accumulated.

Algorithm 1 `sum` — Jacobian initialisation (sparsity + index map)

Require: Child Jacobian J_x in CSR format; rows to sum: $\{r_0, \ldots, r_{s-1}\}$
Ensure: Output Jacobian J with one row per output element; array `idx_map`

1: *cols* $\leftarrow$ union of column indices across summed rows of J_x
2: Sort *cols*; remove duplicates ▷ $O(\texttt{nnz} \log \texttt{nnz})$
3: Set J.`indptr`, J.`indices` from *cols*
4: Build reverse map `col_to_pos`[ ]: for each unique column c at position p, set `col_to_pos`$[c] \leftarrow p$
5: **for** each nonzero k in J_x (across summed rows) **do**
6: `idx_map`$[k] \leftarrow$ `col_to_pos`$[J_x.\texttt{indices}[k]]$
7: **end for**

Evaluation (Algorithm 2) At each parameter update, `eval_jacobian` zeroes the output values and then scatters the child's nonzeros through the precomputed map. The loop body is a single indexed addition per nonzero—no sorting, no allocation, and no matrix multiply.

Algorithm 2 `sum` — Jacobian evaluation

Require: Child Jacobian J_x (values updated); precomputed `idx_map`
Ensure: Output Jacobian J with accumulated values

1: $J.\texttt{data} \leftarrow \mathbf{0}$
2: **for** $k = 0$ **to** $\texttt{nnz}(J_x) - 1$ **do**
3: $J.\texttt{data}[\texttt{idx_map}[k]]$ += $J_x.\texttt{data}[k]$
4: **end for**

Complexity Initialisation costs $O(\texttt{nnz} \log \texttt{nnz})$ (dominated by the sort) and runs once. Evaluation costs $O(\texttt{nnz})$ with a single pass over the child's nonzeros. While sparse matrix multiplication $\mathbf{1}^T J_x$ is also $O(\texttt{nnz})$ in principle, it must allocate a new result matrix, compute

its sparsity pattern, and sort indices at every call—overhead that the `idx_map` approach eliminates entirely. Figure 3.1 illustrates the CSR transformation for a concrete example.

3.3.3 The `left_matmul` atom and Kronecker avoidance

Left multiplication computes $Y = A\,X$ where $A \in \mathbf{R}^{m\times n}$ is a constant (or parameter) matrix and $X \in \mathbf{R}^{n\times p}$ is the child expression. Writing the operation in vectorised form gives

$$\mathbf{vec}(Y) = (I_p \otimes A)\ \mathbf{vec}(X), \tag{3.1}$$

so the Jacobian with respect to $\mathbf{vec}(X)$ is the block-diagonal matrix $I_p \otimes A$ of size $mp \times np$. Forming this Kronecker product explicitly would require storing $p \cdot \mathtt{nnz}(A)$ entries in the best case; worse, composing it with the child's Jacobian via sparse matrix multiplication is $O(p^2 \cdot \mathtt{nnz}(A))$ in general.

The engine avoids the Kronecker product entirely by exploiting the block-diagonal structure. It stores only A (of size $m \times n$) and applies it independently to each of the p column-blocks of the child's Jacobian. The computation is carried out in CSC (Compressed Sparse Column) format, because the child's Jacobian columns each decompose naturally into p consecutive blocks of n rows.

Initialisation During `jacobian_init`, the child's Jacobian sparsity is converted to CSC. For each column j and each block $b \in \{0, \ldots, p-1\}$, the engine identifies which rows of A have overlapping column support with the child's nonzeros in block b. These overlaps determine the output sparsity without ever forming $I_p \otimes A$.

Evaluation (Algorithm 3) At each parameter update, the child's Jacobian values are converted to CSC, then each output nonzero is computed as a sparse dot product of a row of A with the corresponding block-slice of the child's column.

Complexity Let $\bar{k}$ denote the average number of nonzeros per column of J_{child} within a single block. Each output nonzero requires a sparse dot product of cost at most $O(\mathtt{nnz_row}(A) + \bar{k})$. The total work is therefore $O(\mathtt{nnz}(J) \cdot (\mathtt{nnz_row}(A) + \bar{k}))$, which for typical sparse problems is far smaller than the $O(p^2 \cdot \mathtt{nnz}(A))$ cost of forming and multiplying the Kronecker product.

Algorithm 3 `left_matmul` — Jacobian evaluation (block-diagonal multiply)

Require: Constant matrix $A \in \mathbf{R}^{m \times n}$; child Jacobian J_x in CSC with p column-blocks of n rows; precomputed output sparsity J in CSC
Ensure: Output Jacobian J with filled values

Convert J_x values from CSR to CSC ▷ pattern already fixed
for each column j of J **do**
  for each block $b = 0, \ldots, p-1$ **do**
    $S \leftarrow$ nonzeros of J_x column j in rows $[b \cdot n,\ (b+1) \cdot n)$
    for each output nonzero $(b \cdot m + i,\ j)$ in J **do**
      $J.\texttt{data}[\cdot] \leftarrow \text{sparse_dot}(A[i, :],\ S)$
    end for
  end for
end for
Convert J values from CSC to CSR

3.3.4 The `index` atom and structural row selection

Indexing selects a subset of elements from the child expression: $y = x[\text{indices}]$. Its Jacobian is a row-selection matrix P whose rows are drawn from the identity—row i of P is row indices$[i]$ of I. Since the child's Jacobian is stored in CSR, selecting rows amounts to copying contiguous segments of the `indices` and `data` arrays guided by `indptr`. No arithmetic is performed at all.

Initialisation (Algorithm 4) During `jacobian_init`, the output Jacobian's sparsity pattern is constructed by copying the column-index segment of each selected row from the child's Jacobian. The row pointer array `indptr` is built incrementally.

Algorithm 4 `index` — Jacobian initialisation (structural copy)

Require: Child Jacobian J_x in CSR; selection array $\texttt{idx}[0 \ldots k-1]$
Ensure: Output Jacobian J in CSR with selected rows

$J.\texttt{indptr}[0] \leftarrow 0$
for $i = 0$ **to** $k - 1$ **do**
  $r \leftarrow \texttt{idx}[i]$ ▷ source row in J_x
  $\ell \leftarrow J_x.\texttt{indptr}[r+1] - J_x.\texttt{indptr}[r]$
  Copy $J_x.\texttt{indices}[J_x.\texttt{indptr}[r] \ldots J_x.\texttt{indptr}[r] + \ell)$ into $J.\texttt{indices}[J.\texttt{indptr}[i] \ldots]$
  $J.\texttt{indptr}[i+1] \leftarrow J.\texttt{indptr}[i] + \ell$
end for
$J.\texttt{nnz} \leftarrow J.\texttt{indptr}[k]$

Evaluation (Algorithm 5) At each parameter update, `eval_jacobian` copies the `data` segments for the same selected rows. The sparsity pattern is identical to the child's selected rows, so only values move.

Algorithm 5 `index` — Jacobian evaluation (value copy)

Require: Child Jacobian J_x (values updated); selection array $\texttt{idx}[0 \dots k-1]$; output Jacobian J (pattern fixed)
Ensure: Output Jacobian J with copied values
1: **for** $i = 0$ **to** $k - 1$ **do**
2: $r \leftarrow \texttt{idx}[i]$
3: $\ell \leftarrow J.\texttt{indptr}[i+1] - J.\texttt{indptr}[i]$
4: Copy $J_x.\texttt{data}[J_x.\texttt{indptr}[r] \dots J_x.\texttt{indptr}[r] + \ell)$ into $J.\texttt{data}[J.\texttt{indptr}[i] \dots]$
5: **end for**

Complexity Both initialisation and evaluation cost $O\Big(\sum_{i=0}^{k-1} \texttt{nnz}(\text{row}_{\texttt{idx}[i]})\Big)$, proportional to the total number of nonzeros in the selected rows. The operation involves no arithmetic—only `memcpy` calls—making it one of the cheapest atoms in the engine.

The preceding atom deep-dives illustrate the common pattern: each atom transforms the child's CSR arrays directly, using precomputed index mappings to eliminate runtime allocation and sparsity discovery. The remaining sections address the system-level concerns that complete the engine design.

3.4 Memory management

The two-phase design described in Section 3.1 enables tight control over memory allocation. This section describes the four techniques the engine uses to minimise memory overhead.

Exact-size allocation The init phase determines the exact number of nonzeros in every node's Jacobian before any numerical computation occurs. All `malloc` calls therefore allocate arrays of exactly the required size—no over-allocation, no reallocation, and no amortised-doubling strategies. For a tree with N nodes, init performs at most $3N$ allocations (one each for `indptr`, `indices`, and `data` per node), and eval performs zero.

Affine Jacobian caching Nodes whose children are all constants (no variable or parameter dependence) produce Jacobians that never change. The engine detects such nodes during init and marks them as *cached*: their `eval_jacobian` is called once, and subsequent evaluations skip them entirely. In typical problems, constant subexpressions (data matrices, fixed offsets)

constitute a significant fraction of the tree, so caching eliminates a proportional share of eval-time work.

No Python object overhead Because the engine is implemented in C, there is no `scipy.sparse` object creation, no Python reference counting or garbage collection pressure during evaluation, and no format-conversion copies between CSR and CSC. Each Jacobian is stored as three raw C arrays and a set of dimension fields. The total per-node overhead is a fixed-size `struct` plus the three arrays, compared to the multiple Python objects, type metadata, and indirection layers in a `scipy.sparse.csr_matrix`.

Memory footprint analysis The engine's total memory consumption for a single expression tree is:

$$M = \sum_{i=1}^{N} \Big((n_i{+}1)\cdot\texttt{sizeof(int)} + \texttt{nnz}_i \cdot (\texttt{sizeof(int)} + \texttt{sizeof(double)}) \Big) + N\cdot\texttt{sizeof(expr)},$$

where n_i is the number of rows and $\texttt{nnz}_i$ the number of nonzeros in node i's Jacobian. In `CVXPY`'s approach, each sparse matrix multiplication at node i temporarily allocates an additional result matrix, so the peak memory includes both the child's matrix and the product. The engine avoids this by writing directly into the pre-allocated output, so peak memory equals the sum above with no temporary overhead.

3.5 Parameter handling

When a problem contains parameters (symbolic placeholders whose numerical values change between solves), the engine must update the coefficient matrices without repeating the init phase. The two-phase architecture makes this straightforward.

Parameter lifecycle Parameters are registered during problem construction via `register_params`, which associates each parameter node in the expression tree with a pointer to the user-supplied value storage. When the user updates parameter values and requests a new solve, `update_params` copies the new values into the parameter nodes.

Lazy refresh via `jacobian_called` Each node carries a Boolean flag `jacobian_called`. After `update_params` is called, the engine resets this flag on every node that depends (transitively) on a changed parameter. The subsequent eval traversal then recomputes only the

affected nodes. Nodes that depend solely on constants retain their cached Jacobians and are not revisited.

Direct re-traversal Unlike `CVXPY`'s tensor-based parametric approach, which stores a three-dimensional structure mapping each parameter to its coefficient contributions, the engine simply re-traverses the expression tree with the new parameter values. This avoids the storage overhead of the tensor representation and reuses the same tree-walking infrastructure used for the initial evaluation.

Contrast with the tensor approach `CVXPY`'s `TensorView` maintains a dictionary of dictionaries: for each variable, for each parameter, a separate sparse matrix. For problems with p parameters and v variables, this multiplies the storage by up to $p \cdot v$. The engine's approach stores only one Jacobian per node regardless of the number of parameters. The trade-off is that a parameter update requires a full eval traversal rather than a single sparse matrix substitution, but since eval is dominated by $O(\texttt{nnz})$ scatter and copy operations, this cost is modest in practice.

CHAPTER 4 BENCHMARK RESULTS

CHAPTER 5 Benchmark results

CHAPTER 6 CONCLUSION

Texte / Text.

6.1 Synthèse des travaux / Summary of Works

Texte / Text.

6.2 Limitations de la solution proposée / Limitations

6.3 Améliorations futures / Future Research

Texte / Text.

REFERENCES

[1] R. Fourer and M. J. Harrison, "A modern approach to computer systems for linear programming," 1978.

[2] *IBM Mathematical Programming System Extended/370 (MPSX/370) Program Reference Manual*, 2nd ed., IBM Corporation, 1976.

[3] W. Orchard-Hays, "History of mathematical programming systems," *Annals of the History of Computing*, vol. 6, no. 3, pp. 296–312, 1984.

[4] I. Marceau and T. W. Mason, "A matrix-generator system for linear programming problems," Center for Advanced Computation, University of Illinois at Urbana-Champaign, Urbana, IL, Tech. Rep. CAC Document No. 4, 1971. [Online]. Available: https://hdl.handle.net/2142/32405

[5] *MaGen Reference Manual*, Haverly Systems, Inc., Denville, NJ, 1977.

[6] *GAMMA 3.4 Programmer Reference*, Sperry Univac Computer Systems, St. Paul, MN, 1977.

[7] *MIPS III DATAFORM User Manual*, Ketron, Inc., Arlington, VA, 1975.

[8] R. Fourer, "Modeling languages versus matrix generators for linear programming," *ACM Trans. Math. Softw.*, vol. 9, no. 2, pp. 143–183, Jun. 1983. [Online]. Available: https://doi.org/10.1145/357456.357457

[9] A. Brooke, D. Kendrick, and A. Meeraus, *GAMS: A User's Guide.* Redwood City, CA: The Scientific Press, 1988.

[10] J. Bisschop, *AIMMS optimization modeling*, 2006.

[11] R. Fourer, D. M. Gay, and B. W. Kernighan, "A modeling language for mathematical programming," *Management Science*, vol. 36, no. 5, pp. 519–554, 1990.

[12] D. M. Gay, "Writing .nl files," Sandia National Laboratories, Tech. Rep. SAND2005-7907P, 2005, describes AMPL's expression DAG and coefficient extraction internals.

[13] N. Karmarkar, "A new polynomial-time algorithm for linear programming," *Combinatorica*, vol. 4, no. 4, pp. 373–395, 1984.

[14] Y. Nesterov and A. Nemirovskii, *Interior-Point Polynomial Algorithms in Convex Programming*. Philadelphia: Society for Industrial and Applied Mathematics, 1994.

[15] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[16] M. ApS, *MOSEK optimization suite 11.0*, 2026.

[17] ——, "MOSEK modeling cookbook," 2026.

[18] B. O'Donoghue, E. Chu, N. Parikh, and S. Boyd, "Conic optimization via operator splitting and homogeneous self-dual embedding," *Journal of Optimization Theory and Applications*, vol. 169, no. 3, pp. 1042–1068, June 2016. [Online]. Available: http://stanford.edu/~boyd/papers/scs.html

[19] B. O'Donoghue, "Operator splitting for a homogeneous embedding of the linear complementarity problem," *SIAM Journal on Optimization*, vol. 31, pp. 1999–2023, August 2021.

[20] J. Löfberg, "YALMIP: A toolbox for modeling and optimization in MATLAB," in *2004 IEEE international conference on robotics and automation (IEEE Cat. No. 04CH37508)*. IEEE, 2004, pp. 284–289.

[21] W. E. Hart, J.-P. Watson, and D. L. Woodruff, "Pyomo: modeling and solving mathematical programs in Python," *Mathematical Programming Computation*, vol. 3, no. 3, pp. 219–260, 2011.

[22] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[23] I. Dunning, J. Huchette, and M. Lubin, "JuMP: A modeling language for mathematical optimization," *SIAM Review*, vol. 59, no. 2, pp. 295–320, 2017.

[24] M. Lubin, O. Dowson, J. D. Garcia, J. Huchette, B. Legat, and J. P. Vielma, "Jump 1.0: Recent improvements to a modeling language for mathematical optimization," 2023. [Online]. Available: https://arxiv.org/abs/2206.03866

[25] B. Legat, O. Dowson, J. D. Garcia, and M. Lubin, "Mathoptinterface: a data structure for mathematical optimization problems," 2020. [Online]. Available: https://arxiv.org/abs/2002.03447

[26] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.

[27] C. Coey, L. Kapelevich, and J. P. Vielma, "Solving natural conic formulations with Hypatia.jl," *INFORMS Journal on Computing*, vol. 34, no. 5, pp. 2686–2699, 2022.

[28] ——, "Performance enhancements for a generic conic interior point algorithm," *Mathematical Programming Computation*, vol. 15, no. 1, pp. 53–101, 2022.

[29] D. Cederberg and S. Boyd, "Projections onto spectral matrix cones," 2025. [Online]. Available: https://arxiv.org/abs/2511.01089

[30] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006.

[31] M. Grant, "Disciplined Convex Programming," Ph.D. dissertation, Stanford University, Dec. 2004, ph.D. dissertation. [Online]. Available: https://web.stanford.edu/~boyd/papers/pdf/mcg_thesis.pdf

[32] M. Grant, S. Boyd, and Y. Ye, "Disciplined convex programming," in *Global Optimization: From Theory to Implementation*, L. Liberti and N. Maculan, Eds. New York: Springer, 2006, pp. 155–210.

[33] I. CVX Research, "CVX: Matlab software for disciplined convex programming, version 2.0," https://cvxr.com/cvx, Aug. 2012.

[34] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[35] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, "A rewriting system for convex optimization problems," *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

[36] A. Fu, B. Narasimhan, and S. Boyd, "Cvxr: An r package for disciplined convex optimization," *Journal of Statistical Software*, vol. 94, no. 14, 2020. [Online]. Available: http://dx.doi.org/10.18637/jss.v094.i14

[37] M. Udell, K. Mohan, D. Zeng, J. Hong, S. Diamond, and S. Boyd, "Convex optimization in Julia," in *2014 first workshop for high performance technical computing in dynamic languages*. IEEE, 2014, pp. 18–28.

[38] A. Agrawal, S. Diamond, and S. Boyd, "Disciplined geometric programming," *Optimization Letters*, vol. 13, pp. 961–976, 2019.

[39] A. Agrawal and S. Boyd, "Disciplined quasiconvex programming," *Optimization Letters*, vol. 14, pp. 1643–1657, 2020.

[40] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and Z. Kolter, "Differentiable convex optimization layers," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[41] E. Chu, N. Parikh, A. Domahidi, and S. Boyd, "Code generation for embedded second-order cone programming," in *2013 European Control Conference (ECC)*, 2013, pp. 1547–1552.

[42] A. H. Gebremedhin, F. Manne, and A. Pothen, "What color is your Jacobian? Graph coloring for computing derivatives," *SIAM Review*, vol. 47, no. 4, pp. 629–705, 2005.

[43] U. Naumann, "Optimal accumulation of Jacobian matrices by elimination methods on the dual computational graph," *Mathematical Programming*, vol. 99, no. 3, pp. 399–421, 2004.

[44] ——, "Optimal jacobian accumulation is NP-complete," in *Mathematical Programming*, vol. 112, no. 2. Springer, 2008, pp. 427–441.

[45] M. Schaller, G. Banjac, S. Diamond, A. Agrawal, B. Stellato, and S. Boyd, "Embedded code generation with cvxpy," *IEEE Control Systems Letters*, vol. 6, pp. 2653–2658, 2022.

APPENDIX A DÉMO

Texte de l'annexe A. Remarquez que la phrase précédente se termine par une lettre majuscule suivie d'un point. On indique explicitement cette situation à LaTeX afin que ce dernier ajuste correctement l'espacement entre le point final de la phrase et le début de la phrase suivante.

APPENDIX B ENCORE UNE ANNEXE / ANOTHER APPENDIX

Texte de l'annexe B en mode «landscape».

APPENDIX C UNE DERNIÈRE ANNEXE / THE LAST APPENDIX

Texte de l'annexe C.